

Lecture 21: Practice Problems — Technical Case Studies

 Jupyter [questions.ipynb](#)

PDF [questions.pdf](#)

Solutions [View Solutions](#)

 Language: English | [हिंदी / Hinglish](#) |  [View Complete Solutions](#) |  [Overview](#)

Case Study 1: Computer Vision 2D Affine Transformations & Homogeneous Coordinates

Context & Engineering Problem

In modern computer vision and data augmentation pipelines (such as PyTorch `torchvision.transforms` and Spatial Transformer Networks), image geometric perturbations are represented via linear transformations in 2D projective space $\mathbb{P}^2$. Representing 2D points in homogeneous coordinates $\tilde{\mathbf{x}} = [x, y, 1]^T$ allows non-linear translations to be expressed as linear matrix multiplications in $\mathbb{R}^{3 \times 3}$.

An autonomous robot camera captures an object defined by polygon vertices:

$$V = \begin{bmatrix} 0 & 2 & 2 & 0 & 0 & 0 & 3 & 3 & 1 & 1 & 1 \end{bmatrix}$$

The data augmentation pipeline requires applying three sequential geometric operations:

1. **Rotation:** Counter-clockwise rotation by angle $\theta = 45^\circ = \frac{\pi}{4}$ around the origin.
2. **Shear:** Horizontal shearing with shear factor $k_x = 0.5$.
3. **Translation:** Translation by offset vector $\mathbf{t} = [3, -2]^T$.

Mathematical & Implementation Tasks

1. **Construct Homogeneous Elementary Matrices:** Write down the explicit 3×3 transformation matrices for rotation $R(\theta)$, horizontal shear $Sh(k_x)$, and translation $T(t_x, t_y)$.

2. **Matrix Composition & Non-Commutativity:** Compute the composite transformation matrix $M_1 = T \cdot Sh \cdot R$. Then compute an alternate ordering $M_2 = R \cdot T \cdot Sh$. Prove numerically that matrix multiplication does not commute ($M_1 \neq M_2$) and explain why transformation order is critical in vision pipelines.
3. **Determinant & Area Scaling:** Compute the determinant of the 2×2 linear block of M_1 . Prove mathematically that translation does not alter the area scaling factor, and compute the ratio of the transformed polygon area to the original area.
4. **Inverse Mapping (Backward Warping):** In digital image warping, forward mapping causes disocclusion holes (unmapped destination pixels). State why backward warping via M_1^{-1} resolves this problem. Compute M_1^{-1} explicitly.
5. **Python Pipeline:** Implement a Python function to transform arbitrary 2D polygons using homogeneous coordinates and visualize original vs transformed shapes.

Case Study 2: Support Vector Machine (SVM) Maximum-Margin Hyperplane & Vector Projections

Context & Engineering Problem

In statistical machine learning, Support Vector Machines find the optimal linear separating hyperplane $H = \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{w}^T \mathbf{x} + b = 0\}$ that separates two classes while maximizing the geometric margin γ .

Consider a 2D classification dataset with training samples:

- Class +1: $\mathbf{x}_1 = [1, 2]^T$, $\mathbf{x}_2 = [2, 3]^T$
- Class -1: $\mathbf{x}_3 = [3, 1]^T$, $\mathbf{x}_4 = [4, 2]^T$

The decision boundary is defined by parameter vector $\mathbf{w} = [w_1, w_2]^T$ and scalar bias $b \in \mathbb{R}$.

Mathematical & Implementation Tasks

1. **Normal Vector Orthogonality Proof:** Prove that the weight vector $\mathbf{w}$ is strictly orthogonal to the hyperplane H .
2. **Orthogonal Point-to-Hyperplane Distance:** Using vector projection along the unit normal $\hat{\mathbf{w}} = \frac{\mathbf{w}}{|\mathbf{w}|_2}$, derive the formula for the orthogonal signed Euclidean distance from any arbitrary point $\mathbf{x}_0$ to the hyperplane:

$$d(\mathbf{x}_0, H) = \frac{\mathbf{w}^T \mathbf{x}_0 + b}{|\mathbf{w}|_2}$$

3. **Margin Derivation:** Show that if the canonical support vectors satisfy $\mathbf{w}^T \mathbf{x}_+ + b = +1$ and $\mathbf{w}^T \mathbf{x}_- + b = -1$, the total geometric margin between the two bounding hyperplanes is:

$$\gamma = \frac{2}{\|\mathbf{w}\|_2}$$

4. **Analytical Parameter Determination:** Given support vectors $\mathbf{x}_1 = [1, 2]^T$ and $\mathbf{x}_3 = [3, 1]^T$, analytically calculate the optimal weight vector $\mathbf{w}^*$, bias b^* , and the margin width γ .
5. **Python Visualization:** Implement the optimal hyperplane, margin boundaries, and support vectors in Python.

Case Study 3: Quantitative Finance Risk & Sensor PCA Eigendecomposition

Context & Engineering Problem

A quantitative hedge fund tracks the daily returns of $p = 4$ correlated financial asset classes (Equities, Bonds, Commodities, FX) over $N = 500$ trading days. The empirical covariance matrix of centered asset returns $\mathbf{X}_c \in \mathbb{R}^{N \times 4}$ is:

$$\mathbf{\Sigma} = \begin{bmatrix} 4.0 & 2.0 & 1.0 & 0.0 & 2.0 & 3.0 & 0.5 & 0.5 & 1.0 & 0.5 & 2.0 & 0.0 & 0.0 & 0.5 & 0.0 & 1.0 \end{bmatrix}$$

To minimize portfolio risk and eliminate multi-collinearity, the quantitative research team applies Principal Component Analysis (PCA) to find uncorrelated synthetic factor portfolios.

Mathematical & Implementation Tasks

- Symmetry & Positive Semi-Definiteness:** Prove mathematically that any sample covariance matrix $\mathbf{\Sigma} = \frac{1}{N-1} \mathbf{X}_c^T \mathbf{X}_c$ is symmetric and positive semi-definite. State why all its eigenvalues must be real and non-negative ($\lambda_i \geq 0$).
- Lagrangian Formulation of Maximum Variance:** Prove using Lagrange multipliers that the unit direction vector $\mathbf{q}_1 \in \mathbb{R}^p$ ($\|\mathbf{q}_1\|_2 = 1$) that maximizes the variance $\text{Var}(\mathbf{X}_c \mathbf{q}_1) = \mathbf{q}_1^T \mathbf{\Sigma} \mathbf{q}_1$ is the eigenvector of $\mathbf{\Sigma}$ corresponding to the largest eigenvalue λ_1 :

$$\max_{\|\mathbf{q}_1\|=1} \mathbf{q}_1^T \mathbf{\Sigma} \mathbf{q}_1 = \lambda_1$$

- Eigendecomposition & Variance Spectrum:** Compute the 4 eigenvalues $\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq \lambda_4$ of $\mathbf{\Sigma}$. Calculate the total variance $\text{Tr}(\mathbf{\Sigma})$ and the Proportion of Explained Variance (PEV_k) for each principal factor.

4. **Dimension Reduction Criterion:** Determine the minimum number of principal components required to preserve at least 80 of total portfolio risk/variance.
5. **Low-Rank Covariance Approximation:** Compute the rank-2 reconstructed covariance matrix $\Sigma_2 = \lambda_1 \mathbf{q}_1 \mathbf{q}_1^T + \lambda_2 \mathbf{q}_2 \mathbf{q}_2^T$. Compute the Frobenius reconstruction error $\|\Sigma - \Sigma_2\|_F$.